📚 Complete Project Learning Guide

User Role Management Portal — Everything Explained

> This guide teaches you **every concept** used in your project, from the very basics to how each file and function works. Read it top to bottom like a textbook.

□ PART 1: THE BASICS (Concepts You Need First)

Chapter 1: What is C#?

C# (pronounced "C-sharp") is a **programming language** created by Microsoft. Think of it like English is a language humans use to communicate — C# is the language we use to tell the computer what to do.

Key C# Concepts Used in Your Project

1.1 Classes — The Blueprint

A **class** is like a blueprint for creating objects. Just like an architect's blueprint describes what a house looks like, a class describes what data something holds and what it can do.

```
// This is a class - it's a blueprint for a "User"
public class User
{
    public string FullName { get; set; }  // A piece of data (property)
    public string Email { get; set; }     // Another piece of data
}
```

In your project: User.cs and Role.cs are classes that describe what a User and Role look like.

1.2 Properties — The Data Holders

Properties are **variables that belong to a class**. They store data.

```
public string FullName { get; set; }
//  │       │          │    │
//  │       │          │    └── "set" = you CAN change it
//  │       │          └─── "get" = you CAN read it
//  │       └── The name of this property
//  └── The type (string = text)
```

Common types in your project: | Type | What it stores | Example | |------|---------------|---------| | `string` | Text | `"Rushil Parikh"` | | `int` | Whole number | `1`, `42` | | `DateTime` | Date and time | `2026-06-01 10:30:00` | | `bool` | True or false | `true`, `false` |

1.3 Namespaces — Organizing Code

Namespaces are like **folders for your code**. They group related classes together.

```
namespace UserRolePortal.Models  // All model classes live here
{
    public class User { ... }
    public class Role { ... }
}
```

1.4 `using` Statements — Importing Tools

When you write `using SomeLibrary;` at the top of a file, you're saying "I want to use tools from this library."

```
using Microsoft.EntityFrameworkCore;  // "I want database tools"
using System.Security.Claims;         // "I want security tools"
```

It's like saying "I need a hammer" before you can use one.

1.5 `async` / `await` — Waiting for Slow Things

Some operations take time (like reading from a database). Instead of freezing the entire app while waiting, `async`/`await` lets the app keep running.

```
// WITHOUT async - the app FREEZES while reading the database
var user = _context.Users.FirstOrDefault(u => u.Username == "rushil");

// WITH async - the app keeps running while waiting
var user = await _context.Users.FirstOrDefaultAsync(u => u.Username ==
"rushil");
```

Simple analogy: You order food at a restaurant. Instead of standing at the counter and blocking everyone (synchronous), you sit at your table and the waiter brings the food when it's ready (async).

1.6 Lambda Expressions (=>) — Shortcut Functions

The => arrow creates a tiny unnamed function right where you need it.

```
// Full version:
users.Where(delegate(User u) { return u.RoleId == 1; })

// Lambda version (same thing, much shorter):
users.Where(u => u.RoleId == 1)
//           │     └── "where the user's RoleId equals 1"
//           └──── "u" is a temporary name for each user being checked
```

Chapter 2: What is .NET MVC?

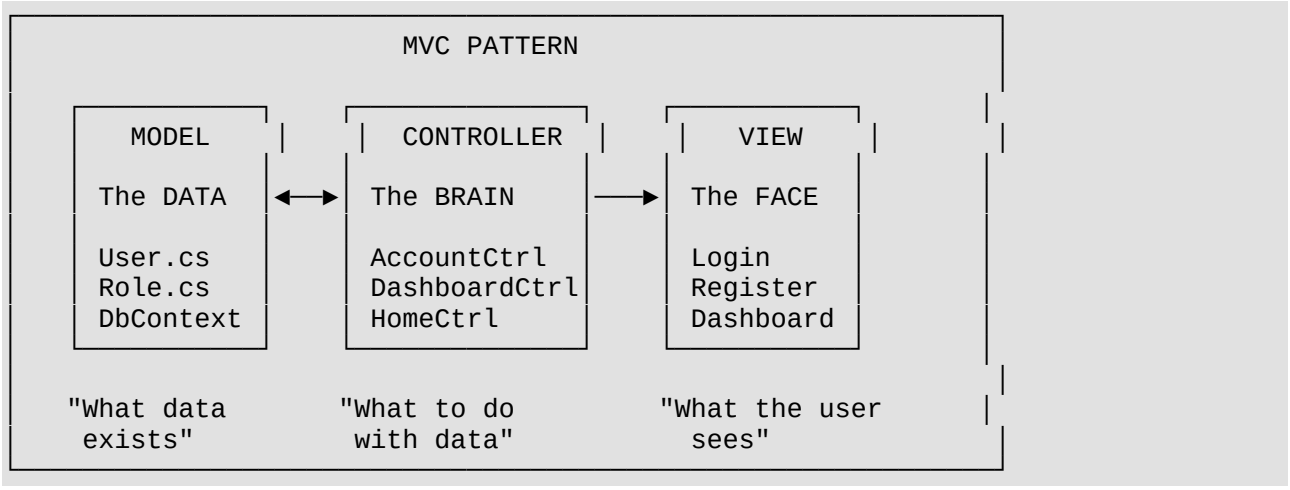
2.1 What is .NET?

.NET is a **framework** — a collection of pre-built tools and libraries that help you build applications without writing everything from scratch.

Think of it like LEGO bricks. Instead of carving each brick yourself, .NET gives you thousands of ready-made pieces to build with.

2.2 What is MVC?

MVC stands for **Model-View-Controller**. It's a **design pattern** — a way of organizing your code into 3 parts:



Real-life analogy:

- **Model** = The kitchen ingredients and recipes (data)
- **Controller** = The chef who decides what to cook (logic/decisions)
- **View** = The plated dish served to the customer (what you see on screen)

2.3 How MVC Works Step by Step

When a user visits `http://localhost/Account/Login`:

```
Step 1: Browser sends request → "I want /Account/Login"
        ↓
Step 2: .NET routing reads the URL
        /Account = Controller name (AccountController)
        /Login   = Action name (Login method)
        ↓
Step 3: AccountController.Login() runs
        - It may read data from Model (database)
        - It decides what View to show
        ↓
Step 4: The Login.cshtml View renders HTML
        ↓
Step 5: HTML is sent back to the browser
        ↓
Step 6: User sees the login page!
```

Chapter 3: What is PostgreSQL?

3.1 The Concept

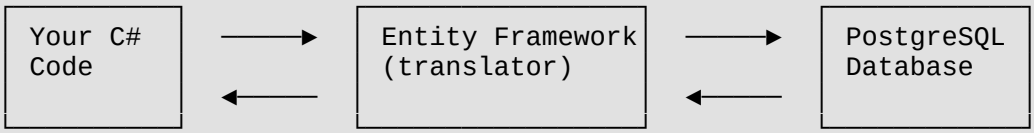
PostgreSQL (often called "Postgres") is a **database** — a structured place to store data permanently. Without a database, all your data disappears when you restart the app.

Think of it like an **Excel spreadsheet** but much more powerful:

```
┌──────────────────────────────────────────────┐
│  TABLE: Roles                                │
└──────────────────────────────────────────────┘

┌──────────┬──────────┐
│ RoleId   │ RoleName │
├──────────┼──────────┤
│ 1        │ Admin    │
│ 2        │ User     │
└──────────┴──────────┘

┌──────────────────────────────────────────────┐
│  TABLE: Users                                │
└──────────────────────────────────────────────┘

┌──────────┬──────────┬──────────┬──────────┬──────────┐
│ UserId   │ FullName │ Username │ Email    │ RoleId   │
├──────────┼──────────┼──────────┼──────────┼──────────┤
│ 1        │ Rushil   │ rushil06 │ r@g.com  │ 1        │
│ 2        │ John     │ john123  │ j@g.com  │ 2        │
└──────────┴──────────┴──────────┴──────────┴──────────┘
```

3.2 How Your Project Connects to PostgreSQL

Your project uses **Entity Framework Core** (EF Core) as a "translator" between C# and PostgreSQL.




```
// You write C#:
var user = await _context.Users.FirstOrDefaultAsync(u => u.Username ==
"rushil");

// EF Core converts it to SQL for PostgreSQL:
// SELECT * FROM "Users" WHERE "Username" = 'rushil' LIMIT 1;
```

Why EF Core? You don't have to write raw SQL queries. You write C#, and EF Core translates it to SQL automatically.

3.3 Connection String — The Address of Your Database

In appsettings.json:

```
"DefaultConnection":
"Host=localhost;Port=5432;Database=URP;Username=postgres;Password=Rushil@06"
```

Part	Meaning
`Host=localhost`	Database is on THIS computer
`Port=5432`	PostgreSQL's default door number
`Database=URP`	The database name is "URP"
`Username=postgres`	Login as "postgres" user
`Password=Rushil@06`	With this password

Chapter 4: What is Entity Framework Core?

4.1 DbContext — The Database Bridge

`DbContext` is your **gateway to the database**. Every time you want to read, create, update, or delete data, you go through the DbContext.

```
// Your DbContext in ApponDbContext.cs:
public class ApponDbContext : DbContext
{
    public DbSet<Role> Roles { get; set; }  // This maps to the "Roles" table
    public DbSet<User> Users { get; set; }  // This maps to the "Users" table
}
```

`DbSet<User>` = "a collection of User rows in the database." When you write `_context.Users`, you're accessing the Users table.

4.2 Migrations — Version Control for Your Database

Migrations are like **save points** for your database structure. When you change a model (add a field, change a type), you create a migration that tells PostgreSQL "here's what changed."

```
Your Model Changes  →  dotnet ef migrations add MyMigration  →  Migration File Created
                    →  dotnet ef database update             →  PostgreSQL Updated!
```

Your project has 2 migrations:

1. **InitialCreate** — Created the Roles and Users tables for the first time
2. **UpdatedUserModel** — Added max-length constraints to columns (e.g., Username max 50 chars)

4.3 CRUD Operations via EF Core

Operation	C# Code	What it does
Create	`_context.Users.Add(user)`	Adds a new row to Users table
Read	`_context.Users.ToListAsync()`	Gets all rows from Users table
Update	`user.FullName = "New Name"` then `SaveChangesAsync()`	Changes a row
Delete	`_context.Users.Remove(user)` then `SaveChangesAsync()`	Removes a row

> [!IMPORTANT] Changes are NOT saved to the database until you call `await _context.SaveChangesAsync()`. Think of it like editing a Word document — changes aren't permanent until you hit "Save."

Chapter 5: Authentication & Authorization

5.1 Authentication = "Who are you?"

When you log in with username and password, the system verifies your identity. This is **authentication**.

5.2 Authorization = "What are you allowed to do?"

After knowing who you are, the system checks what you're allowed to access. Admins can see the User List page; regular Users cannot. This is **authorization**.

5.3 Cookie Authentication (Used in Your Project)

Your project uses **cookies** to remember logged-in users:

```
1. User logs in with correct credentials
2. Server creates a "cookie" (a small data file) containing user info
3. Cookie is stored in the user's browser
4. Every future request sends this cookie automatically
5. Server reads the cookie and knows "oh, this is Rushil, he's an Admin"
```

5.4 Claims — Info Stored in the Cookie

Claims are pieces of information about the user stored in the cookie:

```
var claims = new List<Claim>
{
    new Claim(ClaimTypes.Name, user.Username),     // "Username = rushil06"
    new Claim("FullName", user.FullName),           // "FullName = Rushil
Parikh"
    new Claim(ClaimTypes.Role, "Admin"),             // "Role = Admin"
    new Claim("UserId", user.UserId.ToString())      // "UserId = 1"
};
```

Later, anywhere in the app, you can read these claims:

```
string name = User.FindFirst("FullName")?.Value;  // Gets "Rushil Parikh"
bool isAdmin = User.IsInRole("Admin");              // Gets true/false
```

5.5 Password Hashing — Security

Passwords are **never stored as plain text**. They're converted to a random-looking string called a "hash":

```
Original:  "MyPassword123"
Hashed:    "AQAAAAEAACcQAAAAEH7x2N0K8p..." (unreadable gibberish)
```

You **cannot convert a hash back** to the original password. To verify a login, you hash the entered password and compare the two hashes.

□ PART 2: YOUR PROJECT STRUCTURE

Chapter 6: The Folder Map

```
User Role Management Portal/
└── UserRolePortal/
    ├── UserRolePortal.slnx          ← Solution file (groups projects together)
    └── UserRolePortal/              ← The actual project
        ├── Program.cs               ← ▫ THE STARTING POINT (app starts here)
        ├── appsettings.json         ← ⚙ Configuration (DB connection,
settings)
        ├── UserRolePortal.csproj    ← 📦 Package list (what libraries to
download)
        │
        ├── Models/                  ← 📊 DATA LAYER (what data looks like)
        │   ├── User.cs              ← User data blueprint
        │   ├── Role.cs              ← Role data blueprint
        │   ├── ApponDbContext.cs     ← Database connection bridge
        │   ├── LoginViewModel.cs    ← Login form data shape
        │   ├── UserUpdatedDto.cs    ← Edit-user form data shape
        │   └── ErrorViewModel.cs    ← Error page data shape
        │
        ├── Controllers/             ← ▫ LOGIC LAYER (decisions & actions)
        │   ├── HomeController.cs    ← Handles home page
        │   ├── AccountController.cs ← Handles login, register, logout
        │   └── DashboardController.cs ← Handles dashboard & user management
        │
        ├── Views/                   ← 👁 DISPLAY LAYER (what users see)
        │   ├── _ViewImports.cshtml  ← Global imports for all views
        │   ├── _ViewStart.cshtml    ← Default layout for all views
        │   ├── Home/
        │   │   ├── Index.cshtml     ← Home page (landing page)
        │   │   └── Privacy.cshtml   ← Privacy page
        │   ├── Account/
        │   │   ├── Login.cshtml     ← Login form page
        │   │   └── Register.cshtml  ← Registration form page
        │   ├── Dashboard/
        │   │   ├── Index.cshtml     ← Dashboard page (after login)
        │   │   └── UserList.cshtml  ← User management table (admin only)
        │   └── Shared/
        │       ├── _Layout.cshtml        ← Dashboard layout (sidebar + topbar)
        │       ├── _AuthLayout.cshtml    ← Login/Register layout (clean)
        │       ├── _PublicLayout.cshtml  ← Home page layout (public)
        │       ├── _Layout.cshtml.css    ← CSS for dashboard layout
        │       ├── Error.cshtml          ← Error page
        │       └── _ValidationScriptsPartial.cshtml ← Form validation scripts
        │
        ├── wwwroot/                 ← ▫ STATIC FILES (CSS, JS, images)
        │   ├── css/
        │   │   ├── site.css         ← Dashboard styling
        │   │   ├── custom.css       ← Login/Register/Home styling
        │   │   └── dropdown.css     ← Custom dropdown styling
        │   ├── js/
        │   │   ├── site.js          ← General JavaScript
        │   │   └── custom-dropdown.js ← Custom dropdown functionality
        │   └── favicon.ico          ← Browser tab icon
        │
        └── Migrations/              ← ▫ DATABASE VERSION HISTORY
            ├── InitialCreate.cs     ← First migration: created tables
            └── UpdatedUserModel.cs  ← Second migration: added constraints
```

□ PART 3: EVERY FILE EXPLAINED IN DETAIL

Chapter 7: Configuration Files

7.1 UserRolePortal.csproj — The Package List

What it is: This XML file tells .NET "what framework to use" and "what external packages to download."

Think of it like a **shopping list** for your project.

```
<Project Sdk="Microsoft.NET.Sdk.Web">        <!-- "This is a web project" -->

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>  <!-- "Use .NET 10" -->
    <Nullable>enable</Nullable>                  <!-- "Enable null safety" -->
    <ImplicitUsings>enable</ImplicitUsings>      <!-- "Auto-import common
namespaces" -->
  </PropertyGroup>

  <ItemGroup>
    <!-- Package 1: EF Core Design tools (for creating migrations) -->
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design"
Version="10.0.8" />

    <!-- Package 2: PostgreSQL driver for EF Core -->
    <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL"
Version="10.0.1" />
  </ItemGroup>

</Project>
```

Package	Why you need it
`EntityFrameworkCore.Design`	Lets you run `dotnet ef migrations add` commands
`Npgsql.EntityFrameworkCore.PostgreSQL`	The "translator" that lets EF Core talk to PostgreSQL specifically

---

7.2 appsettings.json — App Configuration

What it is: A JSON file that stores settings your app needs. It's like a **settings page** for your application.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Host=localhost;Port=5432;Database=URP;Username=postgres;Password=Rushil@06"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",          // Log info-level messages
```

```
      "Microsoft.AspNetCore": "Warning"  // Only log warnings from framework
    }
  },
  "AllowedHosts": "*"  // Accept requests from any domain
}
```

The Connection String is the most important part — it tells your app how to find and connect to the PostgreSQL database.

7.3 _ViewImports.cshtml — Global View Settings

```
@using UserRolePortal              // Makes all classes in this namespace
available
@using UserRolePortal.Models       // Makes Model classes (User, Role) available
in views
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers  // Enables special HTML
attributes
```

Tag Helpers are the magic that lets you write:

```
<a asp-controller="Account" asp-action="Login">Login</a>
<!-- Instead of manually writing: -->
<a href="/Account/Login">Login</a>
```

The `asp-controller` and `asp-action` attributes are Tag Helpers — they automatically generate the correct URL.

7.4 _ViewStart.cshtml — Default Layout

```
@{
    Layout = "~/Views/Shared/_Layout.cshtml";
}
```

This says: **"Unless a view says otherwise, use `_Layout.cshtml` as the wrapper."** Individual views can override this (and they do — Login uses `_AuthLayout`, Home uses `_PublicLayout`).

Chapter 8: Program.cs — The Starting Point

Program.cs is **THE FIRST FILE that runs** when you start the application. It sets up everything.

Think of it as the **opening ceremony** of your app. It:

1. Registers all the services (tools) the app needs
2. Configures how the app behaves
3. Starts listening for web requests

Line-by-Line Breakdown:

```
// STEP 1: IMPORTS — Load the tools we need
```

```
using Microsoft.EntityFrameworkCore;                    // Database tools
using UserRolePortal.Models;                             // Our model classes
using Microsoft.AspNetCore.Authentication.Cookies;       // Cookie login tools

// STEP 2: CREATE THE APP BUILDER
var builder = WebApplication.CreateBuilder(args);
// Think of this as "I'm preparing to build my app"
```

Registering Services (Lines 8–43)

Services are **tools that your app needs**. You register them once here, and they're available everywhere.

```
// SERVICE 1: MVC — "I want to use Controllers and Views"
builder.Services.AddControllersWithViews();

// SERVICE 2: HTTP Context Accessor — "I want to access the current user in
views"
builder.Services.AddHttpContextAccessor();

// SERVICE 3: Database — "Connect to PostgreSQL using this connection string"
builder.Services.AddDbContext<ApponDbContext>(options =>

options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")
));
// This reads "DefaultConnection" from appsettings.json

// SERVICE 4: Session — "Remember user data between page loads"
builder.Services.AddDistributedMemoryCache();  // In-memory cache for session
data
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);  // Session expires after 30
min idle
    options.Cookie.HttpOnly = true;                  // JavaScript can't access
this cookie
    options.Cookie.IsEssential = true;               // Cookie is required for
the app
});

// SERVICE 5: Cookie Authentication — "Use cookies to track who's logged in"
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationSc
heme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Account/Login";          // If not logged in, go
here
        options.LogoutPath = "/Account/Logout";        // Logout goes here
        options.ExpireTimeSpan = TimeSpan.FromMinutes(30); // Cookie expires in
30 min
        options.Cookie.HttpOnly = true;
        options.Cookie.IsEssential = true;
    });
```

Building & Configuring the App (Lines 45–74)

```
var app = builder.Build();   // "OK, build the app with all those services"

// MIDDLEWARE PIPELINE — the order MATTERS!
// Think of middleware as security checkpoints at an airport

if (!app.Environment.IsDevelopment())
```

```
{
    app.UseExceptionHandler("/Home/Error");  // In production: show nice error
page
    app.UseHsts();                            // Force HTTPS in production
    app.UseHttpsRedirection();               // Redirect HTTP to HTTPS
}
else
{
    app.UseDeveloperExceptionPage();  // In development: show detailed error
info
}

app.UseStaticFiles();    // Checkpoint 1: Serve CSS, JS, images from wwwroot/
app.MapStaticAssets();   // Map static file paths
app.UseRouting();        // Checkpoint 2: Figure out which controller/action to
call
app.UseSession();        // Checkpoint 3: Load session data

app.UseAuthentication(); // Checkpoint 4: "Who is this user?" (read the cookie)
app.UseAuthorization();  // Checkpoint 5: "Is this user allowed here?" (check
roles)

// ROUTING — "How to translate URLs into controller actions"
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
//           │                │               │
//           │                │               └── Optional ID parameter
//           │                └── Default action = Index
//           └── Default controller = Home
// So "/" becomes "/Home/Index"

app.Run();  // "START THE APP AND LISTEN FOR REQUESTS!"
```

> [!IMPORTANT] **Middleware order matters!** `UseAuthentication()` must come before `UseAuthorization()`. You can't check "is this user allowed?" until you know "who is this user?"

Chapter 9: Models — The Data Layer

9.1 Role.cs — The Simplest Model

```
namespace UserRolePortal.Models
{
    public class Role
    {
        public int RoleId { get; set; }              // Primary Key (unique ID)
        public required string RoleName { get; set; } // "Admin" or "User"
    }
}
```

This creates the **Roles table** in PostgreSQL: | RoleId | RoleName | |--------|----------| | 1 | Admin | | 2 | User |

- `required` keyword = you MUST provide a RoleName when creating a Role. You can't leave it blank.

9.2 User.cs — The Main Model

This is the **most important model** — it defines what a user looks like.

```
public class User
{
    public int UserId { get; set; }  // Primary Key — auto-generated by
PostgreSQL
```

Data Annotations (The `[Attributes]` Above Properties)

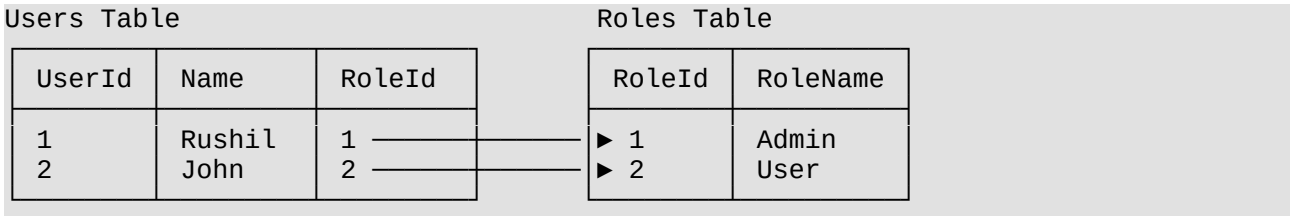
Data annotations are **validation rules**. They ensure the data is correct BEFORE saving to the database:

```
    // FULL NAME: must be 3-100 characters
    [Required(ErrorMessage = "Full Name is required")]        // Can't be empty
    [StringLength(100, MinimumLength = 3,                     // 3 to 100 chars
        ErrorMessage = "Full Name must be between 3 and 100 characters")]
    [Display(Name = "Full Name")]                              // Label shown in
forms
    public required string FullName { get; set; }

    // USERNAME: must be 3-50 characters, must be unique
    [Required(ErrorMessage = "Username is required")]
    [StringLength(50, MinimumLength = 3)]
    public required string Username { get; set; }

    // PASSWORD: at least 6 characters (stored as hash, not plain text)
    [Required(ErrorMessage = "Password is required")]
    [StringLength(100, MinimumLength = 6)]
    public required string Password { get; set; }

    // EMAIL: must be valid email format (contains @ and domain)
    [Required(ErrorMessage = "Email is required")]
    [EmailAddress(ErrorMessage = "Invalid email format")]      // Checks for
valid email
    [StringLength(50)]
    public required string Email { get; set; }

    // MOBILE: exactly 10 digits, no +91 prefix
    [Required(ErrorMessage = "Mobile number is required")]
    [StringLength(10, MinimumLength = 10)]                     // Exactly 10
    [RegularExpression(@"^\d{10}$",                            // Must be 10
digits only
        ErrorMessage = "Mobile number must contain exactly 10 digits")]
    public required string MobileNo { get; set; }

    // DATE OF BIRTH
    [Required]
    [DataType(DataType.Date)]                                   // Show as date
picker
    public DateTime DOB { get; set; }

    // ROLE ID — links to the Roles table (Foreign Key)
    [Required(ErrorMessage = "Role is required")]
    public int RoleId { get; set; }

    // CREATED DATE — set automatically when user registers
    public DateTime? CreatedDate { get; set; }
    // The "?" means this can be null (empty)
```

```
    // GENDER
    [Required(ErrorMessage = "Gender is required")]
    public required string Gender { get; set; }

    // NAVIGATION PROPERTY — links this user to their Role object
    public virtual Role? Role { get; set; }
    // This lets you access user.Role.RoleName (e.g., "Admin")
```

What is a Foreign Key?

`RoleId` in the Users table is a **Foreign Key** — it "points to" the Roles table:



What is a Navigation Property?

`public virtual Role? Role { get; set; }` is a **navigation property**. It lets you "navigate" from a User to their related Role:

```
var user = await _context.Users.Include(u => u.Role).FirstAsync();
// Now you can do:
string roleName = user.Role.RoleName;  // "Admin"
```

Without `.Include(u => u.Role)`, the Role property would be null. `.Include()` tells EF Core to **also load the related Role data**.

9.3 ApponDbContext.cs — The Database Bridge

```
public class ApponDbContext : DbContext   // Inherits from DbContext
{
    // CONSTRUCTOR: receives database connection settings from Program.cs
    public ApponDbContext(DbContextOptions<ApponDbContext> options)
        : base(options)  // Pass settings to parent DbContext class
    { }

    // DbSets = Tables
    public DbSet<Role> Roles { get; set; }  // "Roles" table in PostgreSQL
    public DbSet<User> Users { get; set; }  // "Users" table in PostgreSQL

    // OnModelCreating: runs when EF Core first sets up the database model
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // Define the relationship: One Role → Many Users
        modelBuilder.Entity<User>()
            .HasOne(u => u.Role)         // Each User HAS ONE Role
            .WithMany()                   // Each Role can have MANY Users
            .HasForeignKey(u => u.RoleId); // Connected via RoleId column

        // SEED DATA: Pre-populate the Roles table
        modelBuilder.Entity<Role>().HasData(
```

```
            new Role { RoleId = 1, RoleName = "Admin" },
            new Role { RoleId = 2, RoleName = "User" }
        );
        // This runs during the first migration to insert these two rows
    }
}
```

Dependency Injection (how DbContext gets to controllers):

```
Program.cs registers: builder.Services.AddDbContext<ApponDbContext>(...)
                                              ↓
Controller says: public AccountController(ApponDbContext context)
                                              ↓
.NET automatically creates and passes an ApponDbContext instance
```

9.4 LoginViewModel.cs — Login Form Shape

```
public class LoginViewModel
{
    [Required]
    public string Username { get; set; } = string.Empty;

    [Required]
    public string Password { get; set; } = string.Empty;
}
```

What is a ViewModel? It's a model specifically designed for a View. The Login form only needs Username and Password — not FullName, Email, DOB, etc. So we create a slim model just for the login form.

> [!NOTE] In this project, the Login action actually receives `username` and `password` as individual parameters rather than using this ViewModel. But it's available for future use.

9.5 UserUpdatedDto.cs — Edit Form Shape

```
public class UserUpdatedDto
{
    public int UserId { get; set; }       // Which user to update

    [Required]
    public string FullName { get; set; } = string.Empty;

    [Required, EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required]
    public string MobileNo { get; set; } = string.Empty;

    [Required]
    public DateTime? DOB { get; set; }

    [Required]
    public int RoleId { get; set; }
}
```

What is a DTO? DTO = Data Transfer Object. It's a simple class that carries data from one place to another. When the admin edits a user, the browser sends a JSON object matching this shape. It doesn't include Password or Username because those shouldn't be edited through this form.

Chapter 10: Controllers — The Brain □

10.1 HomeController.cs — The Simplest Controller

```
public class HomeController : Controller  // Inherits from Controller base class
{
    // ACTION 1: Show the home page
    // URL: /Home/Index (or just /)
    public IActionResult Index()
    {
        return View();  // Returns Views/Home/Index.cshtml
    }

    // ACTION 2: Show the privacy page
    // URL: /Home/Privacy
    public IActionResult Privacy()
    {
        return View();  // Returns Views/Home/Privacy.cshtml
    }

    // ACTION 3: Show the error page
    // URL: /Home/Error
    [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore
= true)]
    // ↑ Don't cache this page (always show fresh error info)
    public IActionResult Error()
    {
        return View(new ErrorViewModel { RequestId = Activity.Current?.Id ??
HttpContext.TraceIdentifier });
        // Creates an error model with a unique request ID for debugging
    }
}
```

`return View()` — This tells .NET: "find the `.cshtml` file that matches this controller and action name." So `HomeController.Index()` returns `Views/Home/Index.cshtml`.

10.2 AccountController.cs — Login, Register & Logout

This is the **most important controller** — it handles user authentication.

Constructor — Getting the Database

```
public class AccountController : Controller
{
    private readonly ApponDbContext _context;  // Database access

    // Constructor: .NET automatically injects the database context
    public AccountController(ApponDbContext context)
    {
        _context = context;
```

```
        // Now we can use _context.Users, _context.Roles anywhere in this
controller
    }
```

Registration — GET (Show the Form)

```
    [HttpGet]                          // Only responds to GET requests (opening
the page)
    public IActionResult Register()
    {
        return View();                 // Show Views/Account/Register.cshtml
    }
```

`[HttpGet]` vs `[HttpPost]`:

- **GET** = "I want to SEE something" (loading a page)
- **POST** = "I want to SEND data" (submitting a form)

Registration — POST (Process the Form)

```
    [HttpPost]                                      // Only responds to POST (form
submission)
    [ValidateAntiForgeryToken]                      // Prevents CSRF attacks
(security)
    public async Task<IActionResult> Register(User user, string confirmPassword)
    //     │                                  │              └── Extra parameter
from form
    //     │                                  └── .NET auto-fills this from form
data
    //     └── async because we're doing database work
    {
```

Model Binding: When the form submits, .NET automatically matches form field names to the `User` class properties. The `name="FullName"` input becomes `user.FullName`.

Registration Flow — Step by Step:

```
        // STEP 1: Check if all required fields pass validation
        if (!ModelState.IsValid)
        {
            return View(user);  // Show form again with error messages
        }

        // STEP 2: Check passwords match
        if (user.Password != confirmPassword)
        {
            ModelState.AddModelError("Password", "Passwords do not match");
            return View(user);
        }

        // STEP 3: Check password length
        if (user.Password.Length < 6)
        {
            ModelState.AddModelError("Password", "Password must be at least 6
characters");
            return View(user);
        }

        // STEP 4: Check if username is taken
        var existingUser = await _context.Users
```

```
            .FirstOrDefaultAsync(u => u.Username == user.Username);
        // ↑ "Find the FIRST user whose Username matches, or return null"

        if (existingUser != null)  // Someone already has this username
        {
            ModelState.AddModelError("Username", "Username already exists");
            return View(user);
        }

        // STEP 5: Check if email is taken
        var existingEmail = await _context.Users
            .FirstOrDefaultAsync(u => u.Email == user.Email);

        if (existingEmail != null)
        {
            ModelState.AddModelError("Email", "Email already registered");
            return View(user);
        }

        // STEP 6: Hash the password (NEVER store plain text passwords!)
        var passwordHasher = new PasswordHasher<User>();
        user.Password = passwordHasher.HashPassword(user, user.Password);
        // "MyPassword123" becomes "AQAAAAEAACcQAAAAEH7x2N0K8p..."

        // STEP 7: Set creation date
        user.CreatedDate = DateTime.UtcNow;

        // STEP 8: Convert DOB to UTC (PostgreSQL requirement)
        user.DOB = user.DOB.ToUniversalTime();

        // STEP 9: Add to database
        _context.Users.Add(user);     // "Hey database, here's a new user"
        await _context.SaveChangesAsync();  // "NOW actually save it"

        // STEP 10: Redirect to login page with success message
        TempData["Success"] = "Registration successful! Proceed to login.";
        return RedirectToAction("Login");
        // ↑ Sends the browser to /Account/Login
    }
```

[!TIP] **`TempData`** is a temporary storage that survives ONE redirect. After the redirect, the data is automatically deleted. Perfect for "success" or "error" messages that should only show once.

Login — POST (Verify Credentials)

```
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Login(string username, string password)
    {
        // STEP 1: Null check
        if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))
        {
            ModelState.AddModelError("", "Username and password are required");
            return View();
        }

        // STEP 2: Find user by username (also load their Role)
        var user = await _context.Users
            .Include(u => u.Role)                          // Also load Role
data
            .FirstOrDefaultAsync(u => u.Username == username);
```

```
        // STEP 3: User not found?
        if (user == null)
        {
            ModelState.AddModelError("", "Invalid username or password");
            return View();
            // NOTE: We say "invalid username OR password" — not "username not
found"
            // This is a security best practice (don't reveal which part is
wrong)
        }

        // STEP 4: Verify password
        var passwordHasher = new PasswordHasher<User>();
        var result = passwordHasher.VerifyHashedPassword(user, user.Password,
password);
        //                                                  │           │
│
        //                                                  │           │
└── What user typed
        //                                                  │           └── Hashed
password from DB
        //                                                  └── The user object

        if (result == PasswordVerificationResult.Failed)
        {
            ModelState.AddModelError("", "Invalid username or password");
            return View();
        }

        // STEP 5: Create authentication cookie with Claims
        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.Name, user.Username),        // Store username
            new Claim("FullName", user.FullName),              // Store full
name
            new Claim(ClaimTypes.Role, user.RoleId == 1 ? "Admin" : "User"), //
Store role
            new Claim("UserId", user.UserId.ToString())        // Store user ID
        };
        // "user.RoleId == 1 ? "Admin" : "User""
        // This is a ternary: IF RoleId is 1 THEN "Admin" ELSE "User"

        var claimsIdentity = new ClaimsIdentity(claims,
CookieAuthenticationDefaults.AuthenticationScheme);
        await HttpContext.SignInAsync(
            CookieAuthenticationDefaults.AuthenticationScheme,
            new ClaimsPrincipal(claimsIdentity));
        // ↑ This CREATES the cookie and sends it to the browser

        // STEP 6: Redirect to dashboard
        return RedirectToAction("Index", "Dashboard");
        //                      │          │
        //                      │          └── Controller = DashboardController
        //                      └── Action = Index method
    }
```

Logout

```
    [HttpGet]
    public async Task<IActionResult> Logout()
    {
        await
```

```
HttpContext.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);
        // ↑ DELETES the authentication cookie from the browser

        return RedirectToAction("Index", "Home");
        // Go back to the home page
    }
```

10.3 DashboardController.cs — Dashboard & User Management

```
[Authorize]  // ← PROTECTS THE ENTIRE CONTROLLER
             // Anyone not logged in gets redirected to /Account/Login
public class DashboardController : Controller
{
    private readonly ApponDbContext _context;

    public DashboardController(ApponDbContext context)
    {
        _context = context;
    }
```

`[Authorize]` — This is the **gatekeeper**. Without being logged in, you CANNOT access any page in this controller.

Dashboard Index

```
    [HttpGet]
    public async Task<IActionResult> Index()
    {
        // Read logged-in user's name from claims (cookie data)
        string fullName = User.FindFirst("FullName")?.Value ?? "User";
        ViewBag.FullName = fullName;
        // ↑ ViewBag is a dynamic container to pass data from Controller to View

        // If user is an Admin, load statistics
        if (User.IsInRole("Admin"))
        {
            var users = await _context.Users.Include(u => u.Role).ToListAsync();
            // ↑ Get ALL users with their roles

            ViewBag.TotalUsers = users.Count;                    // e.g., 10
            ViewBag.AdminCount = users.Count(u => u.RoleId == 1); // e.g., 3
            ViewBag.UserCount = users.Count(u => u.RoleId == 2);  // e.g., 7

            // Get the 5 most recent registrations
            ViewBag.RecentUsers = users
                .OrderByDescending(u => u.CreatedDate)  // Newest first
                .Take(5)                                  // Only take 5
                .ToList();
        }

        return View();
    }
```

`ViewBag` — A way to pass data from Controller to View. It's like putting a note in your pocket that the view can read:

```
Controller: ViewBag.FullName = "Rushil";
View:       @ViewBag.FullName → shows "Rushil"
```

User List (Admin Only)

```
    [HttpGet]
    [Authorize(Roles = "Admin")]  // ← ONLY Admins can access this!
    public async Task<IActionResult> UserList()
    {
        List<User> users = await _context.Users
            .Include(u => u.Role)                       // Load Role data too
            .OrderByDescending(u => u.CreatedDate)      // Newest first
            .ToListAsync();                              // Execute the query

        return View(users);  // Pass the user list to the View
        // ↑ "return View(users)" means "here's data for the view to display"
    }
```

Update User (Admin Only)

```
    [HttpPost]
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> UpdateUser([FromBody] UserUpdatedDto dto)
    //                                          └── "Read JSON from the request
body"
    {
        try
        {
            var user = await _context.Users.FindAsync(dto.UserId);
            // ↑ Find user by primary key (UserId)

            if (user == null)
            {
                return Json(new { success = false, message = "User not
found." });
                // ↑ Send JSON response back to the browser
            }

            // Update the user's fields
            user.FullName = dto.FullName;
            user.Email = dto.Email;
            user.MobileNo = dto.MobileNo;
            user.RoleId = dto.RoleId;

            await _context.SaveChangesAsync();  // Save to database

            return Json(new { success = true, message = "User updated
successfully." });
        }
        catch (Exception ex)
        {
            return Json(new { success = false, message = "Error: " +
ex.Message });
        }
    }
```

> [!NOTE] This action uses **`[FromBody]`** and returns **`Json()`** because it's called from JavaScript (AJAX), not a regular form submission. The browser sends JSON data and expects JSON back.

Delete User (Admin Only)

```
    [HttpPost]
    [Authorize(Roles = "Admin")]
```

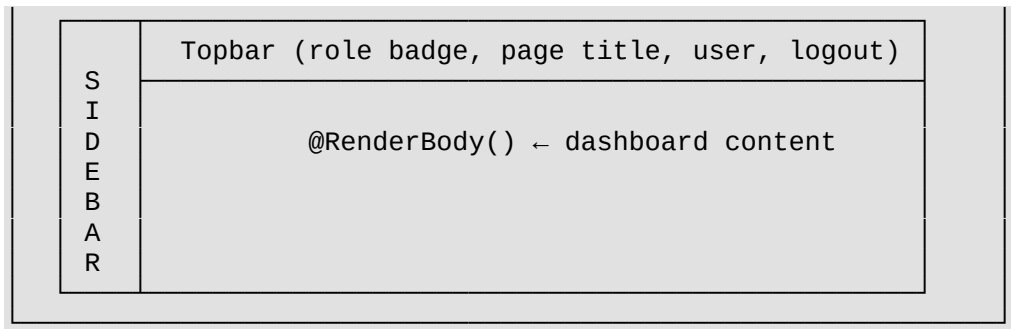
```
    public async Task<IActionResult> DeleteUser(int id)
    {
        try
        {
            var user = await _context.Users.FindAsync(id);
            if (user == null)
            {
                return Json(new { success = false, message = "User not
found." });
            }

            _context.Users.Remove(user);        // Mark for deletion
            await _context.SaveChangesAsync();   // Execute deletion

            return Json(new { success = true, message = "User deleted
successfully." });
        }
        catch (Exception ex)
        {
            return Json(new { success = false, message = "Error: " +
ex.Message });
        }
    }
```

Chapter 11: Views — What Users See 👁

11.1 Layouts — The Page Wrappers

Your project has **3 layouts** for different sections of the app:

```
┌──────────────────────────────────────────────────────────┐
│  _PublicLayout.cshtml        Used by: Home page          │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Topbar (logo + login/register buttons)           │    │
│  │  ┌──────────────────────────────────────────┐    │    │
│  │  │          @RenderBody() ← page content    │    │    │
│  │  └──────────────────────────────────────────┘    │    │
│  │ Footer (contact info, links)                     │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│  _AuthLayout.cshtml          Used by: Login, Register    │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Simple Topbar (logo only)                        │    │
│  │ Blue accent bar                                  │    │
│  │  ┌──────────────────────────────────────────┐    │    │
│  │  │          @RenderBody() ← login/register form │    │
│  │  └──────────────────────────────────────────┘    │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│  _Layout.cshtml              Used by: Dashboard, User List │
```



`@RenderBody()` — This is the **placeholder** where each page's unique content gets inserted. Think of the layout as a picture frame, and `@RenderBody()` is where the picture goes.

11.2 _Layout.cshtml — Dashboard Layout (Key Parts)

```
<!-- SIDEBAR: Shows navigation links -->
<nav id="sidebar">
    <!-- Dashboard link (always visible) -->
    <a class="nav-link @(ViewData["Title"]?.ToString() == "Dashboard" ? "active"
: "")"
       asp-controller="Dashboard" asp-action="Index">
        <i class="fas fa-tachometer-alt"></i>
        <span class="link-text">Dashboard</span>
    </a>

    <!-- User List link (ONLY visible to Admins) -->
    @if (User.IsInRole("Admin"))
    {
        <a class="nav-link" asp-controller="Dashboard" asp-action="UserList">
            <i class="fas fa-users"></i>
            <span class="link-text">User List</span>
        </a>
    }
</nav>
```

`@if (User.IsInRole("Admin"))` — This is **Razor syntax** (C# inside HTML). The `@` symbol switches from HTML to C#. This checks if the logged-in user is an Admin and only shows the User List link for admins.

```
<!-- TOPBAR: Shows user info and actions -->
<header id="topbar">
    <!-- Show role badge -->
    @if(User.IsInRole("Admin"))
    {
        <span>🛡️ Admin</span>
    }
    else
    {
        <span>👤 User</span>
    }

    <!-- Show page title -->
    <h4>@ViewData["Title"]</h4>

    <!-- Show logged-in user's name (from Claims) -->
    <span>@(User.FindFirst("FullName")?.Value ?? "User")</span>
```

```
    <!-- Home and Logout buttons -->
    <a asp-controller="Home" asp-action="Index">Home</a>
    <a asp-controller="Account" asp-action="Logout">Logout</a>
</header>

<!-- PAGE CONTENT inserted here -->
<main>
    @RenderBody()
</main>

<!-- Scripts section (views can add page-specific scripts) -->
@await RenderSectionAsync("Scripts", required: false)
```

`@await RenderSectionAsync("Scripts", required: false)` — This lets individual views add their own JavaScript. The `required: false` means it's optional (not every view needs extra scripts).

11.3 Home/Index.cshtml — Landing Page

```
@{
    ViewData["Title"] = "Home";                                  // Set page
title
    Layout = "~/Views/Shared/_PublicLayout.cshtml";             // Use public
layout
}

<section class="home-hero">
    <div class="home-hero-content">
        <h1>A professional<br><span class="orange">user</span>
management<br>portal</h1>

        <!-- Show different buttons based on login status -->
        @if (User.Identity.IsAuthenticated)
        {
            <!-- Logged in: show Dashboard button -->
            <a asp-controller="Dashboard" asp-action="Index" class="orange-
btn">Dashboard</a>
        }
        else
        {
            <!-- Not logged in: show Register and Login buttons -->
            <a asp-controller="Account" asp-action="Register" class="orange-
btn">Register</a>
            <a asp-controller="Account" asp-action="Login" class="dark-
btn">Login</a>
        }
    </div>
</section>
```

`@if (User.Identity.IsAuthenticated)` — Checks if the user is logged in. This dynamically changes what buttons appear.

11.4 Account/Login.cshtml — Login Form

```
@{
    ViewData["Title"] = "Login";
    Layout = "~/Views/Shared/_AuthLayout.cshtml";   // Use the clean auth layout
```

```
}

<section class="form-page">
    <div class="form-card">
        <h2>User Login</h2>

        <!-- Show success message (e.g., after registration) -->
        @if (TempData["Success"] != null)
        {
            <div class="alert alert-success">
                @TempData["Success"]   <!-- "Registration successful!" -->
            </div>
        }

        <!-- Show error messages (e.g., invalid credentials) -->
        @if (!ViewData.ModelState.IsValid)
        {
            <div class="alert alert-danger">
                <div asp-validation-summary="All"></div>
                <!-- ↑ Shows ALL error messages from ModelState -->
            </div>
        }

        <!-- Login Form -->
        <form asp-controller="Account" asp-action="Login" method="post">
            @Html.AntiForgeryToken()   <!-- Anti-CSRF security token -->

            <div class="form-group">
                <label>Username *</label>
                <input type="text" name="Username" required>
                <!--                    └── This name matches the Login action
parameter -->
            </div>

            <div class="form-group">
                <label>Password *</label>
                <input type="password" name="Password" required>
            </div>

            <button type="submit">Login</button>
        </form>

        <div class="form-footer">
            <a asp-action="Register">Don't have an account? Register</a>
        </div>
    </div>
</section>
```

`@Html.AntiForgeryToken()` — This generates a hidden token that prevents **CSRF attacks** (Cross-Site Request Forgery). It ensures the form was submitted from YOUR website, not a malicious one.

`asp-validation-summary="All"` — This Tag Helper shows all validation errors. When the controller adds `ModelState.AddModelError(...)`, those messages appear here.

11.5 Account/Register.cshtml — Registration Form

This form collects all user data. Key points:

```
<form asp-controller="Account" asp-action="Register" method="post"
id="registerForm">
    @Html.AntiForgeryToken()

    <!-- Each input's "name" attribute must match the User model property names
-->
    <input type="text" name="FullName" required>      <!-- → user.FullName -->
    <input type="email" name="Email" required>        <!-- → user.Email -->
    <input type="text" name="MobileNo" required>      <!-- → user.MobileNo -->
    <input type="date" name="DOB" required>           <!-- → user.DOB -->
    <input type="text" name="Username" required>      <!-- → user.Username -->
    <input type="password" name="Password" required>  <!-- → user.Password -->
    <input type="password" name="confirmPassword">    <!-- → confirmPassword
parameter -->

    <!-- Role dropdown -->
    <select name="RoleId" required>                   <!-- → user.RoleId -->
        <option value="">Select Role...</option>
        <option value="1">Admin</option>
        <option value="2">User</option>
    </select>

    <!-- Gender radio buttons -->
    <input type="radio" name="Gender" value="Male">   <!-- → user.Gender -->
    <input type="radio" name="Gender" value="Female">
    <input type="radio" name="Gender" value="Other">
</form>
```

Client-side validation (JavaScript):

```
// Runs when form is submitted
document.getElementById('registerForm').addEventListener('submit', function(e) {
    var password = document.getElementById('regPassword').value;
    var confirm = document.getElementById('regConfirmPassword').value;
    if (password !== confirm) {
        e.preventDefault();  // STOP the form from submitting
        alert('Passwords do not match!');
    }
});
```

This checks passwords match **before** sending to the server (faster user feedback). The server also checks (defense in depth — check on both sides).

11.6 Dashboard/Index.cshtml — Dashboard

This page shows:

- Welcome message with user's name
- Role badge (Admin/User)
- Current date/time
- **Admin only**: Activity section with recent registrations

```
<!-- Welcome header -->
<h1>Welcome back, @ViewBag.FullName!</h1>

<!-- Role badge -->
<span>@(User.IsInRole("Admin") ? "Admin" : "User")</span>
```

```
<!-- Admin-only: Recent registrations -->
@if (User.IsInRole("Admin"))
{
    @{
        var recentUsers = ViewBag.RecentUsers as
List<UserRolePortal.Models.User>;
        // ↑ Cast ViewBag data to the correct type
    }

    @foreach (var user in recentUsers)
    {
        // Calculate "time ago"
        var diff = DateTime.Now - user.CreatedDate.Value;
        if (diff.TotalMinutes < 1) timeAgo = "Just now";
        else if (diff.TotalMinutes < 60) timeAgo = $"{(int)diff.TotalMinutes}m
ago";
        // ... etc.

        <div>
            <span>@user.FullName</span>
            <span>@(user.RoleId == 1 ? "Admin" : "User")</span>
            <small>@user.Email</small>
            <small>@timeAgo</small>
        </div>
    }
}
```

11.7 Dashboard/UserList.cshtml — User Management Page

This is the **most complex view**. It has:

1. Summary stats cards
2. Search and filter bar
3. User data table
4. Edit modal (popup)
5. Delete modal (popup)
6. JavaScript for search, edit, and delete

```
@model IEnumerable<UserRolePortal.Models.User>
<!-- ↑ This view receives a list of User objects from the controller -->

<!-- Summary Cards -->
<h5>@Model.Count()</h5>                    <!-- Total users -->
<h5>@Model.Count(u => u.RoleId == 1)</h5>  <!-- Admin count -->
<h5>@Model.Count(u => u.RoleId == 2)</h5>  <!-- User count -->

<!-- User Table -->
<table id="userTable">
    @foreach (var user in Model)
    {
        <tr data-role="@(user.RoleId == 1 ? "Admin" : "User")">
            <td>@user.FullName</td>
            <td>@user.Username</td>
            <td>@user.Email</td>
            <td>@user.MobileNo</td>
```

```
            <td>@user.DOB.ToString("dd MMM yyyy")</td>
            <td>@(user.RoleId == 1 ? "Admin" : "User")</td>
            <td>@(user.CreatedDate?.ToString("dd MMM yyyy") ?? "N/A")</td>
            <td>
                <!-- Edit button → opens edit modal -->
                <button onclick="loadUserData(@user.UserId,
'@user.FullName', ...)">
                    Edit
                </button>
                <!-- Delete button → opens delete modal -->
                <button onclick="setDeleteId(@user.UserId, '@user.FullName')">
                    Delete
                </button>
            </td>
        </tr>
    }
</table>
```

JavaScript Functions in UserList:

```
// 1. SEARCH & FILTER — runs on every keystroke and dropdown change
function filterTable() {
    var searchText = document.getElementById('searchInput').value.toLowerCase();
    var roleFilter = document.getElementById('roleFilter').value;
    var rows = document.querySelectorAll('#userTable tbody tr');

    rows.forEach(function(row) {
        var text = row.textContent.toLowerCase();    // All text in the row
        var role = row.getAttribute('data-role');     // "Admin" or "User"
        var matchesSearch = text.includes(searchText);
        var matchesRole = !roleFilter || role === roleFilter;

        row.style.display = (matchesSearch && matchesRole) ? '' : 'none';
        // Show row if BOTH conditions match, hide otherwise
    });
}

// 2. LOAD USER DATA — fills the edit modal with user's current data
function loadUserData(id, fullName, email, mobile, roleId) {
    document.getElementById('editUserId').value = id;
    document.getElementById('editFullName').value = fullName;
    document.getElementById('editEmail').value = email;
    document.getElementById('editMobileNo').value = mobile;
    document.getElementById('editRoleId').value = roleId;
}

// 3. SUBMIT EDIT — sends updated data to server via AJAX
function submitEdit() {
    var formData = {
        UserId: document.getElementById('editUserId').value,
        FullName: document.getElementById('editFullName').value,
        Email: document.getElementById('editEmail').value,
        MobileNo: document.getElementById('editMobileNo').value,
        RoleId: document.getElementById('editRoleId').value
    };

    fetch('/Dashboard/UpdateUser', {       // Send to
DashboardController.UpdateUser
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData)      // Convert JS object to JSON string
    })
    .then(response => response.json())      // Parse server response as JSON
```

```
    .then(data => {
        if (data.success) {
            location.reload();              // Refresh page to see changes
        } else {
            alert(data.message);
        }
    });
}

// 4. SUBMIT DELETE — sends delete request via AJAX
function submitDelete() {
    var id = document.getElementById('deleteUserId').value;

    fetch('/Dashboard/DeleteUser/' + id, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' }
    })
    .then(response => response.json())
    .then(data => {
        if (data.success) {
            location.reload();
        } else {
            alert(data.message);
        }
    });
}
```

`fetch()` — This is the modern JavaScript way to make HTTP requests without reloading the page (AJAX). It sends data to the server and handles the response.

Chapter 12: Migrations — Database Version History

12.1 InitialCreate.cs

This migration created the initial database structure:

```
// UP = "Apply this change"
protected override void Up(MigrationBuilder migrationBuilder)
{
    // Create Roles table
    migrationBuilder.CreateTable(
        name: "Roles",
        columns: table => new
        {
            RoleId = table.Column<int>(nullable: false)  // Primary Key, auto-
increment
                .Annotation("Npgsql:ValueGenerationStrategy",
IdentityByDefaultColumn),
            RoleName = table.Column<string>(nullable: false)
        });

    // Create Users table with Foreign Key to Roles
    migrationBuilder.CreateTable(
        name: "Users",
        columns: table => new
        {
            UserId = table.Column<int>(nullable: false),  // Primary Key
            FullName = table.Column<string>(nullable: false),
```

```
            Username = table.Column<string>(nullable: false),
            Password = table.Column<string>(nullable: false),
            Email = table.Column<string>(nullable: false),
            MobileNo = table.Column<string>(nullable: false),
            DOB = table.Column<DateTime>(nullable: false),
            RoleId = table.Column<int>(nullable: false),     // Foreign Key
            CreatedDate = table.Column<DateTime>(nullable: true),
            Gender = table.Column<string>(nullable: false)
        },
        constraints: table =>
        {
            table.ForeignKey("FK_Users_Roles_RoleId",
                column: x => x.RoleId,
                principalTable: "Roles",
                principalColumn: "RoleId",
                onDelete: ReferentialAction.Cascade);
            // ↑ If a Role is deleted, all Users with that Role are also deleted
        });

    // Insert seed data (Admin and User roles)
    migrationBuilder.InsertData(
        table: "Roles",
        values: new object[,] { { 1, "Admin" }, { 2, "User" } });
}

// DOWN = "Undo this change" (rollback)
protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropTable(name: "Users");
    migrationBuilder.DropTable(name: "Roles");
}
```

12.2 UpdatedUserModel.cs

This migration added **max length constraints** to columns (from `text` to `character varying(50)`):

```
// Changed Username from unlimited text → max 50 characters
migrationBuilder.AlterColumn<string>(
    name: "Username", table: "Users",
    type: "character varying(50)", maxLength: 50);

// Changed Password from unlimited → max 100 characters
// Changed MobileNo from unlimited → max 15 characters
// Changed FullName from unlimited → max 100 characters
// Changed Email from unlimited → max 50 characters
```

□ PART 4: THE COMPLETE FLOW

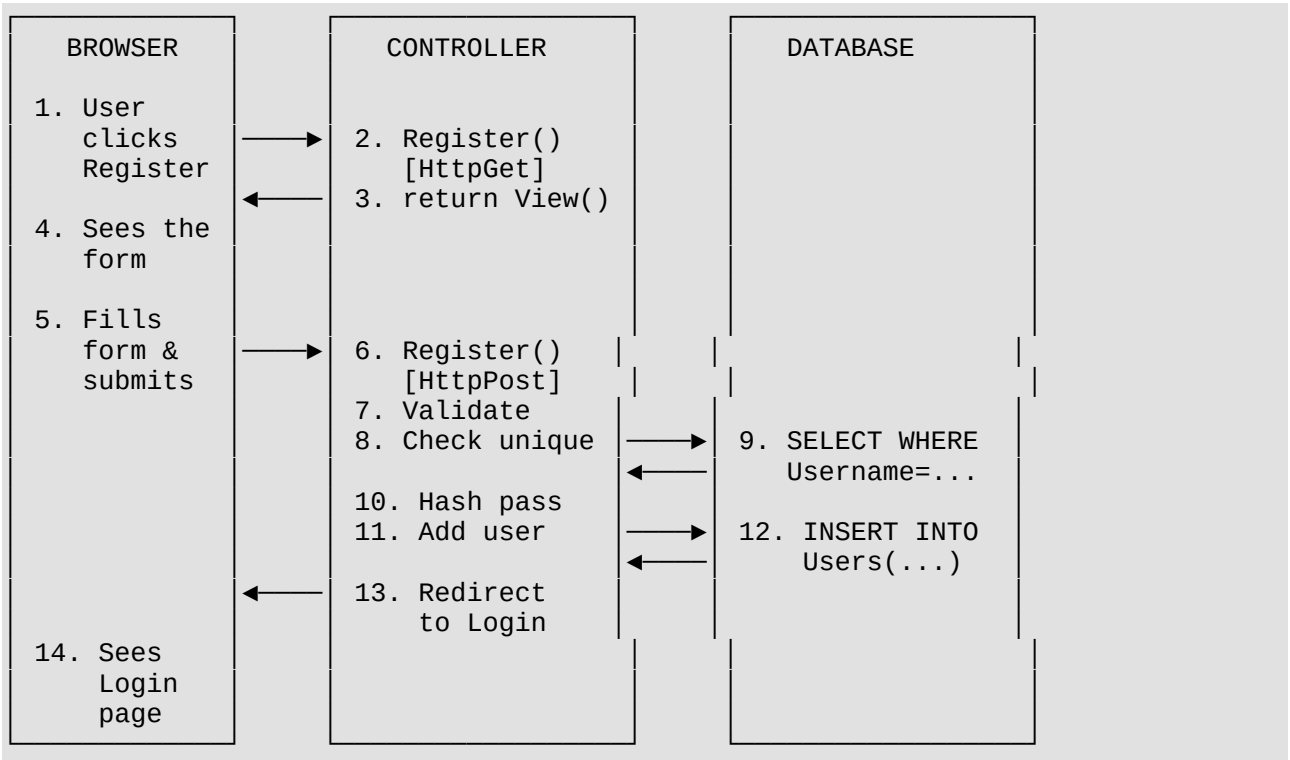
Chapter 13: How Everything Works Together

13.1 Application Startup Flow

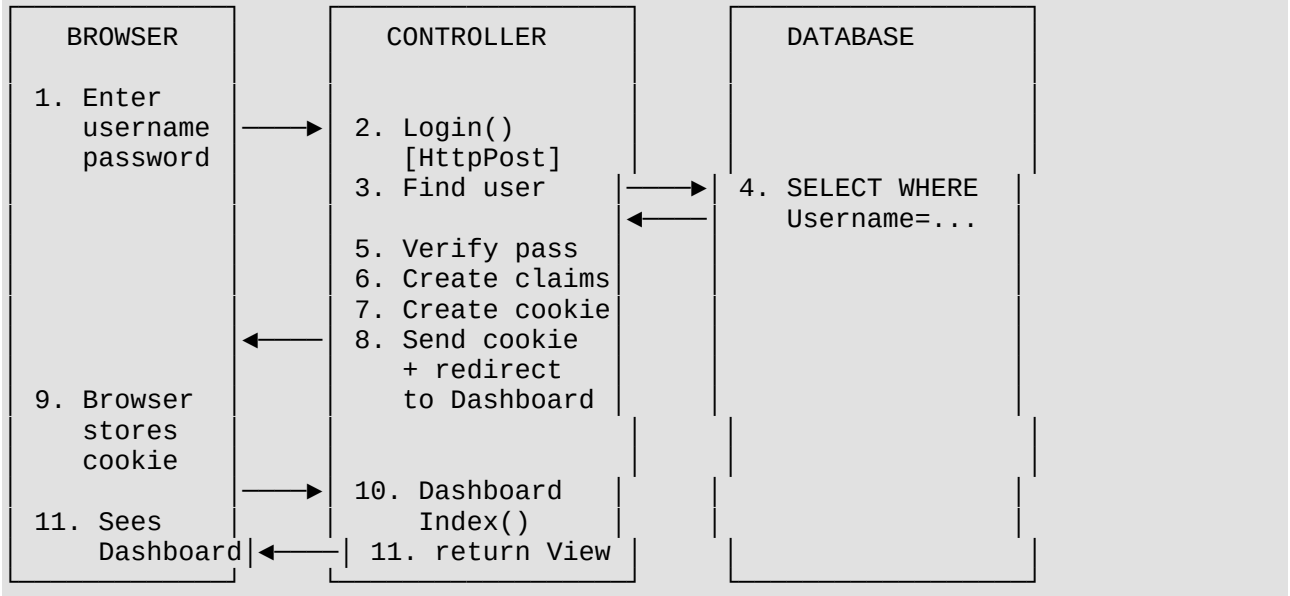
```
1. User runs "dotnet run" or starts from Visual Studio
   ↓
```

```
2. Program.cs executes:
   a. Creates WebApplication builder
   b. Registers services (MVC, Database, Auth, Session)
   c. Builds the app
   d. Configures middleware pipeline
   e. Sets up routing rules
   f. Starts listening on http://localhost:5xxx
   ↓
3. App is ready to receive requests!
```

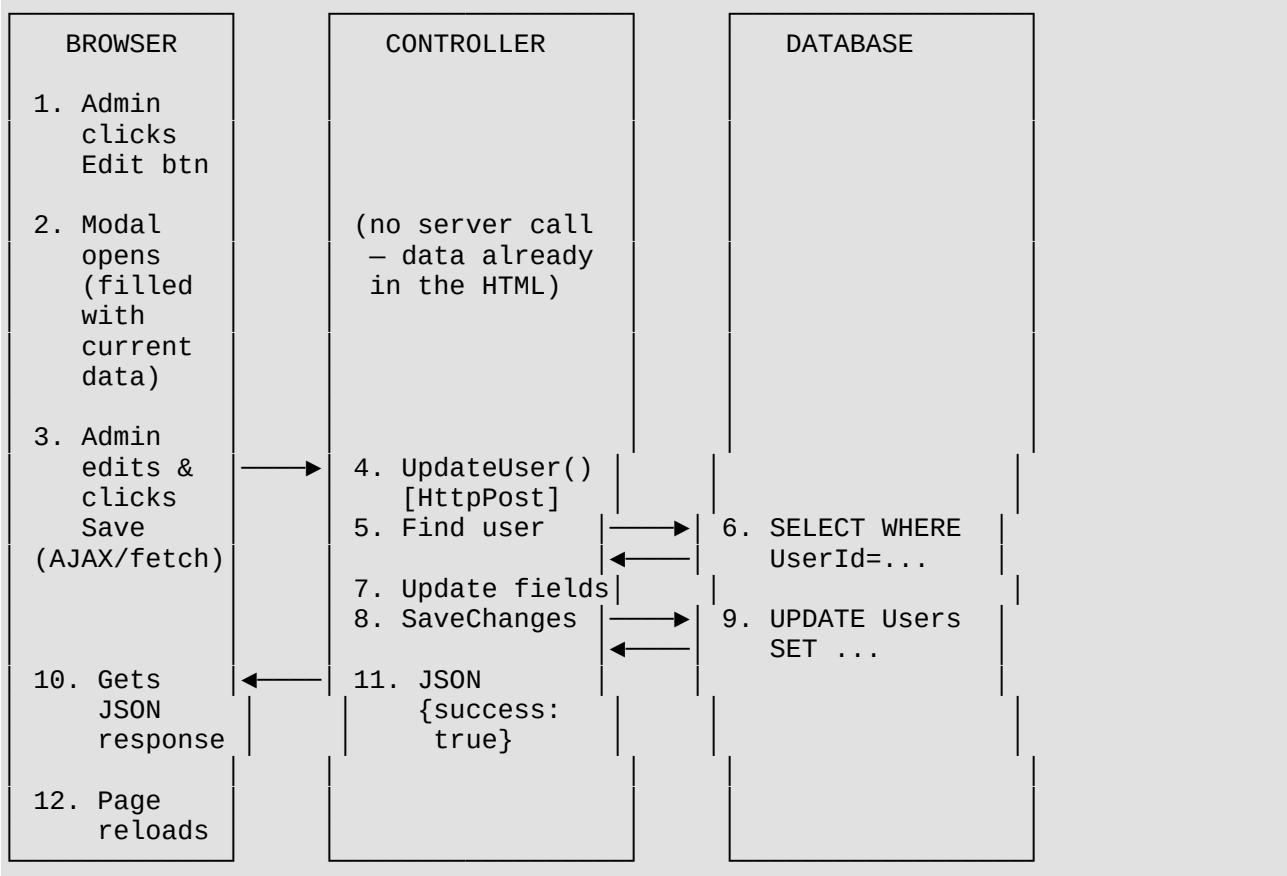
13.2 User Registration Flow



13.3 User Login Flow



13.4 Admin Editing a User Flow



13.5 URL → Controller → View Mapping

URL the user visits	Controller	Action Method	View File
`/` or `/Home/Index`	HomeController	Index()	Views/Home/ Index.cshtml
`/Home/Privacy`	HomeController	Privacy()	Views/Home/ Privacy.cshtml
`/Account/ Register` (GET)	AccountController	Register() [Get]	Views/Account/ Register.cshtml
`/Account/ Register` (POST)	AccountController	Register() [Post]	Redirects to Login
`/Account/Login` (GET)	AccountController	Login() [Get]	Views/Account/ Login.cshtml
`/Account/Login` (POST)	AccountController	Login() [Post]	Redirects to Dashboard
`/Account/Logout`	AccountController	Logout()	Redirects to Home
`/Dashboard/ Index`	DashboardController	Index()	Views/Dashboard/ Index.cshtml
`/Dashboard/ UserList`	DashboardController	UserList()	Views/Dashboard/ UserList.cshtml
`/Dashboard/`	DashboardController	UpdateUser()	Returns JSON

URL the user visits	Controller	Action Method	View File
`UpdateUser` (POST)			
`/Dashboard/DeleteUser/{id}` (POST)	DashboardController	DeleteUser()	Returns JSON

Chapter 14: Security Features in Your Project

Feature	Where	What it protects
Password Hashing	AccountController Register/Login	Passwords are never stored as plain text
`[Authorize]`	DashboardController (class level)	Only logged-in users can access dashboard
`[Authorize(Roles = "Admin")]`	UserList, UpdateUser, DeleteUser	Only admins can manage users
`[ValidateAntiForgeryToken]`	Register, Login POST actions	Prevents CSRF attacks
`@Html.AntiForgeryToken()`	Login.cshtml, Register.cshtml	Generates the anti-CSRF token in forms
Cookie HttpOnly	Program.cs	JavaScript can't read the auth cookie
Cookie IsEssential	Program.cs	Cookie works even if user hasn't consented
`ModelState.IsValid`	Register POST	Validates all data annotations before saving
Duplicate checks	Register POST	Prevents duplicate usernames and emails

---

Chapter 15: Glossary — Quick Reference

Term	Simple Meaning
MVC	Model-View-Controller design pattern
Model	A C# class that represents data (like User, Role)
View	An HTML template (.cshtml) that the user sees
Controller	C# class that handles requests and decides what to do
DbContext	The bridge between C# code and the database
DbSet<T>	Represents a table in the database

Term	Simple Meaning
Migration	A version update for your database structure
Claims	Pieces of info about a user stored in their cookie
Middleware	Code that runs on every request (like security checkpoints)
ViewBag	A way to pass data from Controller to View
TempData	Temporary data that survives one redirect
Tag Helper	Special HTML attributes (like `asp-controller`)
Razor	The engine that lets you write C# inside HTML (`.cshtml`)
DTO	Data Transfer Object — a simple data-carrying class
ViewModel	A model designed specifically for a View
CRUD	Create, Read, Update, Delete operations
AJAX	Making server requests without page reload (using `fetch`)
Foreign Key	A column that links one table to another
Navigation Property	A property that lets you access related data
Dependency Injection	.NET automatically provides services where needed
Seed Data	Pre-loaded data (Admin/User roles inserted during migration)
Hashing	Converting readable text into unreadable gibberish (one-way)
CSRF	An attack where a malicious site submits forms as you

Chapter 16: Static Files (Frontend Magic) □

While C# runs on the server, **CSS and JavaScript** run in the user's browser to make the app look beautiful and interactive. These live in the `wwwroot/` folder.

16.1 CSS (Styling)

- **`site.css`**: Contains general variables (colors, fonts) and styles for the Dashboard layout. It defines the dark theme background `#120f10` and the signature orange accent `#ff4b00`.
- **`custom.css`**: Contains specific styling for the public pages (Home, Login, Register). It handles the layout of the large centered forms, the hero section typography, and hover effects on buttons.

- **`dropdown.css`**: Styles the custom dark-themed dropdowns used across the app. Instead of the standard browser `<select>`, this CSS helps create a beautiful custom dropdown UI.

16.2 JavaScript (Interactivity)

- **`custom-dropdown.js`**: This script automatically finds `<select>` elements and converts them into the beautiful custom UI styled by `dropdown.css`.
- **UserList View JS**: Inside `UserList.cshtml`, there is JavaScript that handles **real-time searching** (filtering table rows as you type) and **AJAX calls** (talking to the server using `fetch()` without reloading the page when editing or deleting a user).

Chapter 17: How to Run the Project Locally ♂

If you ever need to start this project from scratch or run it, here is the exact process:

Step 1: Start PostgreSQL

Ensure your PostgreSQL database server is running on port `5432` and the password matches your `appsettings.json` file.

Step 2: Apply Migrations (If first time)

Open your terminal in the project folder and run:

```
dotnet ef database update
```

This reads your C# migrations and builds the actual tables in PostgreSQL.

Step 3: Run the App

Run this command to start the server:

```
dotnet run
```

This starts the local web server. You will see an output telling you the local URL.

Step 4: Access the Portal

Open your web browser and go to the URL provided.

- Click **Register** to create an Admin account (select Role: Admin).
- Click **Login** with your new credentials.
- Welcome to your Dashboard!

> [!TIP] **How to explore further**: Try changing small things (like a validation message) and see what happens. Break things intentionally to understand what each piece does. The best way to learn is by experimenting!